Interview Preparation (when interview is scheduled)

Four Pillars of OOPS:

1. **Abstraction**: Abstraction is a process of hiding implementation details and exposing only the functionality to the user.

   There are two ways to achieve abstraction in Java:
   - Abstract class
   - Interface

2. **Encapsulation:** Encapsulation is the process of wrapping code and data together into a single unit.

   In order to achieve encapsulation in java:
   - Declare the variables as private
   - Declare the setters and getters to set and get the variable values

3. **Inheritance:** Inheritance is the process of one class inheriting properties and methods from another class in Java. Inheritance is used when we have is-a relationship between objects. Inheritance in Java is implemented using extends keyword.

4. **Polymorphism:** Polymorphism is the ability to perform many things in many ways. There are two types of polymorphism as listed below:
   a. Static or Compile-time Polymorphism
   b. Dynamic or Run-time Polymorphism

Static or Compile-time Polymorphism:

1. when the compiler is able to determine the actual function, it's called compile-time polymorphism.
2. Compile-time polymorphism can be achieved by method overloading in java.
3. When different functions in a class have the same name but different signatures, it's called method overloading.
4. A method signature contains the name and method arguments.

5. So, overloaded methods have different arguments.
6. The arguments might differ in the numbers or the type of arguments.

Dynamic or Run-time Polymorphism:

1. Dynamic (or run-time) polymorphism occurs when the compiler is not able to determine at compile-time which method (superclass or subclass) will be called.
2. This decision is made at run-time.
3. Run-time polymorphism is achieved through method overriding, which happens when a method in a subclass has the same name, return type, and parameters as a method in its superclass.
4. When the superclass method is overridden in the subclass, it is called method overriding.

Types of Inheritance:

Single Inheritance: A subclass inherits from only one superclass.

Example (Java):

```
class Animal {
  void sound () {
      System.out.println("Animal makes a sound");
   }
}
class Dog extends Animal {
   void bark () {
      System.out.println("Dog barks");
   }
}
```

Multilevel Inheritance: A class inherits from a class which itself is derived from another class (a chain of inheritance).

Example:

```
class Animal {
   void sound () {
      System.out.println("Animal makes sound");
   }
}

class Dog extends Animal {
   void bark () {
      System.out.println("Dog barks");
   }
}

class Puppy extends Dog {
   void weep () {
      System.out.println("Puppy weeps");
   }
}
```

Hierarchical Inheritance: Multiple classes inherit from a single superclass.

Example:

```
class Animal {
   void sound() {
```

```
        System.out.println("Animal makes sound");
    }
}

class Dog extends Animal {
    void bark() {
        System.out.println("Dog barks");
    }
}

class Cat extends Animal {
    void meow() {
        System.out.println("Cat meows");
    }
}
```

Multiple Inheritance (Not Supported Directly in Java via Classes): A subclass inherits from more than one superclass.

Java Limitation: Java does not support multiple inheritance with classes due to ambiguity (Diamond Problem). But it supports it through interfaces.

Example with Interfaces:

```
interface Printable {
    void print ();
}
interface Showable {
    void show ();
```

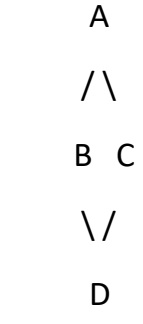
```
    }

    class Demo implements Printable, Showable {
        public void print () {
            System.out.println("Printing");
        }
        public void show () {
            System.out.println("Showing");
    }
}
```

Hybrid Inheritance: A combination of two or more types of inheritance.

Example: Combining hierarchical and multiple inheritance using interfaces.

Diamond Problem: The structure of inheritance forms a diamond shape:



Class B and C both inherit from class A

Class D inherits from both B and C

Now, if both B and C override a method from A, and D calls that method — which version should it use? That's the diamond problem.

Abstract Class:

Java abstract class is a class that cannot be instantiated by itself, it needs to be subclassed by another class to use its properties. An abstract class is declared using the "abstract" keyword in its class definition.

1. An instance of an abstract class cannot be created.
2. Constructors are allowed.
3. We can have an abstract class without any abstract method.
5. There can be a final method in abstract class but any abstract method in class (abstract class) cannot be declared as final .
6. We can define static methods in an abstract class
7. If a class contains at least one abstract method then compulsory should declare a class as abstract

Interface in Java:

In Java, an interface is a reference type, similar to a class, that can contain:

- Abstract methods (no body until Java 7)
- Default methods (Java 8+)
- Static methods (Java 8+)
- Private methods (Java 9+)
- Constants (public static final fields)

String, String Buffer and String Builder:

String:

- Package: java.lang.String
- Mutability: Immutable (Once created, it cannot be changed)
- Thread-Safety: Yes, because it's immutable
- Performance: Slow for string modifications (e.g., +, concat)
- Usage: Best when string content does not change

```
String str = "Hello";
str.concat(" World");   // Returns a new string
System.out.println(str);  // Output: Hello
```

StringBuffer:

- Package: java.lang.StringBuffer
- Mutability: Mutable
- Thread-Safety: Yes (All methods are synchronized)
- Performance: Slower than StringBuilder due to synchronization
- Usage: Use when you need thread-safe string modifications

```
StringBuffer sb = new StringBuffer("Hello");
sb.append(" World");
System.out.println(sb);  // Output: Hello World
```

StringBuilder:

- Package: java.lang.StringBuilder
- Mutability: Mutable
- Thread-Safety: No (Not synchronized)
- Performance: Faster than StringBuffer
- Usage: Preferred for single-threaded environments

```
StringBuilder sb = new StringBuilder("Hello");
sb.append(" World");
System.out.println(sb);  // Output: Hello World
```

Ways of creating Strings:

In Java, there are two primary ways to create a String. Each has differences in memory usage and how Java handles them internally (especially regarding the String Constant Pool):

1. Using String Literals (Recommended)

```
String s1 = "Hello";
String s2 = "Hello"; // Refers to the same object as s1 from the String Pool
```

- Memory: Stored in the String Constant Pool
- Duplicates are avoided: If a string already exists in the pool, the reference is reused.
- Efficient for memory when many strings have the same content.

2. Using new Keyword

```
String s3 = new String("Hello");
```

- Memory: Stored in the Heap, not in the pool (although the "Hello" part still exists in the pool).
- Creates a new object every time, even if the content is the same.
- Used rarely unless explicitly required.
-

```
String s1 = "Java";          // from pool
String s2 = "Java";          // refers to same object
String s3 = new String("Java");  // new object in heap
```

System.out.println(s1 == s2); // true (same reference)

System.out.println(s1 == s3); // false (different references)

System.out.println(s1.equals(s3)); // true (same content)

Difference Between String Constant Pool and Heap in Java:

In Java, String objects can be stored in two places in memory:

1. String Constant Pool (SCP)
2. Heap Memory

These are managed differently and have different behaviours.

1. String Constant Pool (SCP)

- Also called String Intern Pool.
- It is a special memory region inside the heap, but logically separate from the general heap.
- Stores unique literal string values.
- If a string literal already exists in the pool, Java reuses the same object instead of creating a new one.
- Memory-efficient — avoids duplicate string objects.
- Created at compile-time if it's a literal, or using String.intern().

Example:

String s1 = "hello";

String s2 = "hello";

System.out.println(s1 == s2); // true (same object from pool)

2. Heap Memory (Normal Heap)

- General-purpose memory for all Java objects (including Strings created using new).

- Each new keyword creates a new object in heap, even if content is the same.
- Allows duplicate string values.
- More memory is used (unless interned).

◆ Example:

String s3 = new String("hello");

String s4 = new String("hello");

System.out.println(s3 == s4); // false (different objects in heap)

intern() Method – Bridge Between Heap and Pool

If you want to move a heap string to the pool:

String s5 = new String("hello").intern();

System.out.println(s1 == s5); // true

Equals and == for comparison:

1. == Operator in Java

For primitive types (int, float, char, etc.):

- Compares values directly.

int a = 10;

int b = 10;

System.out.println(a == b); // true

For objects:

- Compares references (i.e., whether both variables refer to the same memory location).

String s1 = new String("Java");

String s2 = new String("Java");

System.out.println(s1 == s2); // false (different objects in memory)

2. equals() Method in Java:

1. Defined in the Object class.
2. Used to compare object contents.
3. Can be overridden in custom classes to define how two objects should be considered equal.

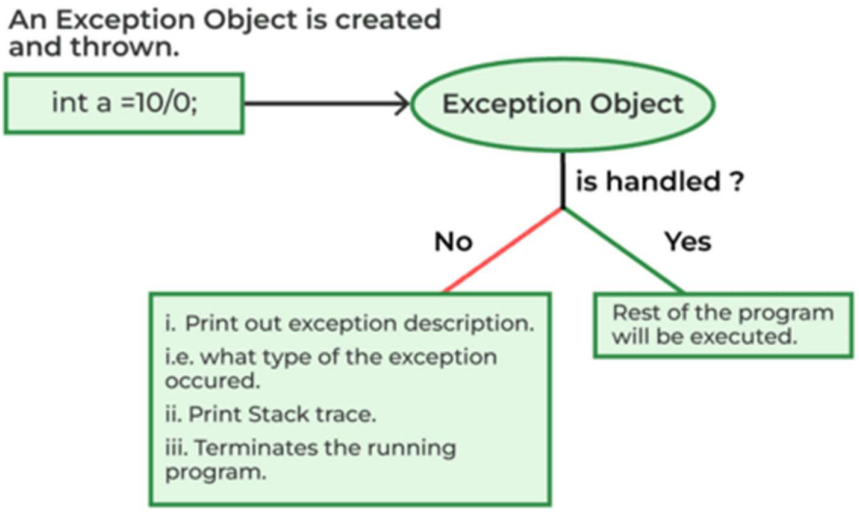
String s1 = new String("Java");

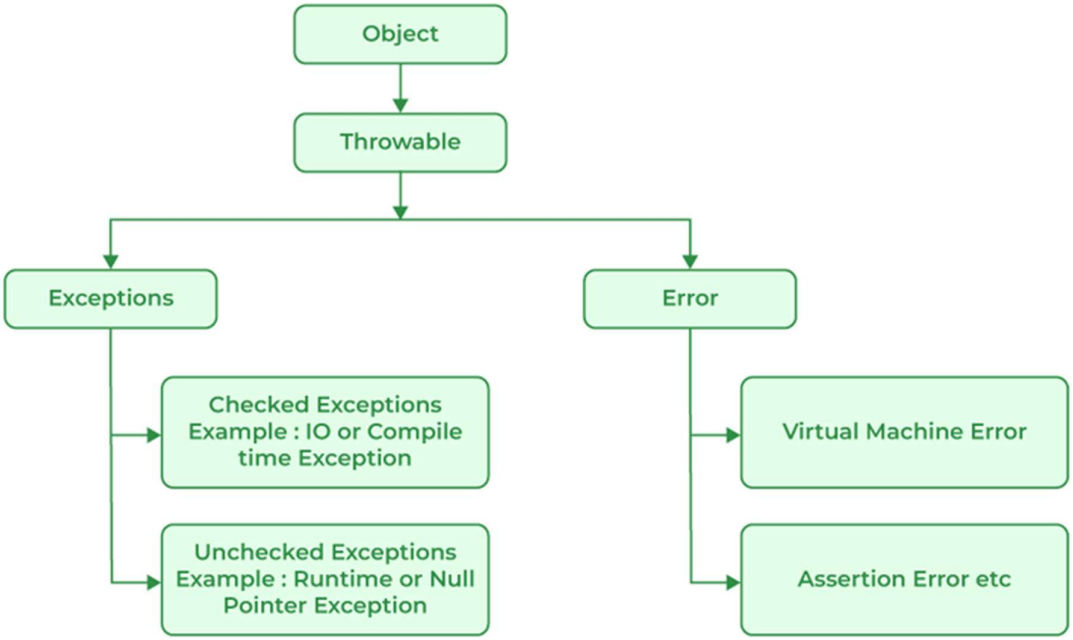
String s2 = new String("Java");

System.out.println(s1.equals(s2)); // true (compares values)

Exception Handling in Java:

1. Exception handling in Java is an effective mechanism for managing runtime errors to ensure the application's regular flow is maintained.
2. Some Common examples of exceptions include ClassNotFoundException, IOException, SQLException, RemoteException, etc.
3. With the help of exception handling, we can detect and handle the exceptions gracefully so that the normal flow of the program can be maintained.
4. The finally block always executes, even if no exception occurs, making it a reliable place for cleanup operations.





Difference Between Exception and Error

Error	Exception
An Error indicates a serious problem that a reasonable application should not try to catch.	Exception indicates conditions that a reasonable application might try to catch

Error	Exception
This is caused by issues with the JVM or hardware.	This is caused by conditions in the program such as invalid input or logic errors.
Examples: OutOfMemoryError, StackOverFlowError	Examples: IOException, NullPointerException

Try-Catch Block:

1. A try-catch block in Java is a mechanism to handle exception.
2. The try block contains code that might throw an exception and the catch block is used to handle the exceptions if it occurs.

```
try {
// Code that may throw an exception
} catch (ExceptionType e) {
// Code to handle the exception
}
```

finally Block:

1. The finally block is used to execute important code regardless of whether an exception occurs or not.
2. finally, block always executes after the try-catch block.
3. It is also used for resource cleanup.

try: The try block contains code that might throw an exception. If an exception occurs within this block, the control is transferred to the corresponding catch block.

catch: The catch block is used to handle the exception thrown by the try block. Multiple catch blocks can be used to handle different types of exceptions.

```
catch (IOException e) {
   // handle IOException
} catch (SQLException e) {
   // handle SQLException
}
```

3. finally: The finally block contains code that will always execute, regardless of whether an exception was thrown or not. It is typically used for cleanup activities, such as closing files or releasing resources.

4.throw: The throw keyword is used to explicitly throw an exception.

5. throws: The throws keyword is used in the method signature to declare that the method might throw exceptions.

Java 8 Features:

1. Lambda Expression
2. Functional interface
3. Method Reference
4. Stream API
5. Optional classes
6. Comparator and Comparable

Lambda Expression

1. A Lambda expression is a simple piece of code that takes in parameters and produces a result.
2. It's primarily used with functional interfaces, which are interfaces with just one abstract method and n number of static or default methods.
3. They're typically defined inline when creating an instance of a functional interface.

Syntax:

```
int sum1 = (a, b) -> (a+b)
```

Functional interface:

1. A functional interface is characterized by having only one abstract method, though it can also include any number of default or static methods.
2. Function interfaces, also known as Single Abstract Method interfaces, are a addition to Java 8, aimed at enabling functional programming.

3. Method Reference:

1. In Java 8, method references provide a concise way to refer to methods or constructors without actually invoking them.
2. They act as shorthand for lambda expressions, allowing you to pass methods as arguments to functional interfaces.
3. For example, the following lambda expression takes a string input and converts it to lowercase using a built-in method from the String class.

```
Function<String, String> func = (str) -> str.toLowerCase();
System.out.println(func.apply("LAMBDA EXPRESSION"));
```

You can replace the above code with method reference as shown below.

```
Function<String, String> func= String::toLowerCase;
System.out.println(func.apply("METHOD REFERENCE"));
```

There are basically 4 types of method references:

1. Static Method: ClassName::staticMethodName
2. Reference to an instance method of a particular object: object::methodName
3. Reference to an instance method of a Class Type: ClassName::methodName
4. Reference to a constructor: ClassName::new

Optional classes:

1. Java introduced the Optional class in java.util package to improve the management of null values.
2. Traditionally, null signifies the absence of a value, but this practice frequently results in NullPointerExceptions (NPEs) when not handled correctly.
3. These exceptions can be difficult to debug and may lead to runtime failures in Java applications.

Comparator and Comparable:

Both Comparator and Comparable are interfaces in Java used for comparing objects, but they serve different purposes.

Comparable:

1. The Comparable interface is used to impose a natural ordering on the objects of a class.
2. If a class implements Comparable, it means that its objects can be ordered automatically.
3. The class implementing Comparable must override the compareTo(Object obj) method that takes a single parameter, which compares this object with another object of the same type and returns a negative integer, zero, or a positive integer depending on whether this object is less than, equal to, or greater than the other object, respectively.

Comparator Interface:

1. The Comparator interface is used to define custom comparison logic for objects that do not implement Comparable or when you want to define multiple ways of comparing objects.
2. The Comparator interface defines a single method called compare(Object o1, Object o2), which compares two objects and returns a negative integer, zero, or a positive integer depending on whether the

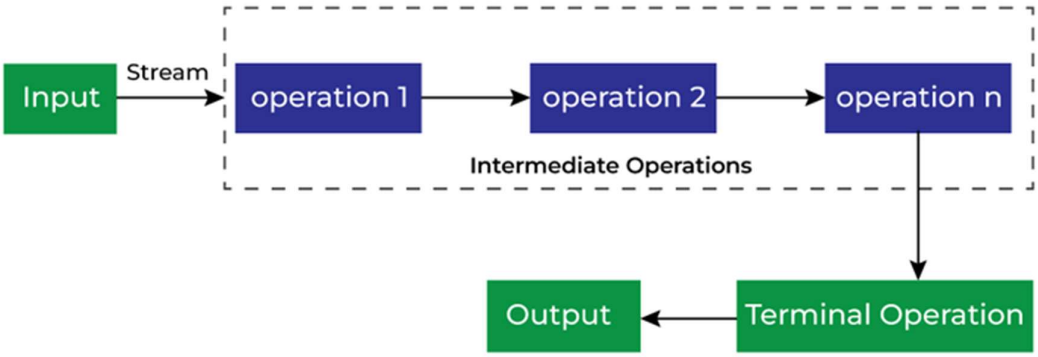
first object is less than, equal to, or greater than the second object, respectively.

3. Using a Comparator allows you to define multiple ways of comparing objects, each with its own criteria.
4. This flexibility is especially useful when you need to sort objects in different ways depending on the context or user preferences.
5. Implementing the Comparable interface requires modifying the class itself to provide a natural ordering.
6. But if you cannot modify the class (e.g., it's a third-party library or part of the Java core classes), using a Comparator allows you to define ordering externally.

Stream API:

1. Stream was introduced in Java 8, the Stream API is used to process collections of objects.
2. A stream in Java is a sequence of objects that supports various methods that can be pipelined to produce the desired result.
3. Stream API is a way to express and process collections of objects.
4. Enable us to perform operations like filtering, mapping, reducing, and sorting.
5. There are two types of Operations in Streams:
    1. Intermediate Operations
    2. Terminal Operations

Intermediate Operations:



1. Intermediate Operations are the types of operations in which multiple methods are chained in a row.
2. Methods are chained together.
3. Intermediate operations transform a stream into another stream.
4. It enables the concept of filtering where one method filters data and passes it to another method after processing.

Terminal Operations

1. Terminal Operations are the type of Operations that return the result. These Operations are not processed further just return a final result value.

Parallel Streams: Streams can be processed sequentially or in parallel for improved performance on multi-core processors.

Creating Streams:

```
List<String> myList = Arrays.asList("apple", "banana", "orange");
Stream<String> sequentialStream = myList.stream();
Stream<String> parallelStream = myList.parallelStream();
```

Creates a stream from an array:

```
int[] numbers = {1, 2, 3, 4, 5};
IntStream stream = Arrays.stream(numbers);
```

Stream.of() method creates a stream from individual values.

```
Stream<String> stream = Stream.of("apple", "banana", "orange");
```

filter(): Filters elements based on a predicate.

```
List<String> filtered = myList.stream()
                        .filter(s -> s.startsWith("a"))
                        .collect(Collectors.toList());
```

map(): Transforms elements using a function.

```
List<Integer> lengths = myList.stream()
                        .map(String::length)
                        .collect(Collectors.toList());
```

sorted(): Sorts elements.

```
List<String> sorted = myList.stream()
                      .sorted() // we can use comparator here as a arg
                      .collect(Collectors.toList());
```

distinct(): Removes duplicate elements.

```
List<String> distinct = myList.stream()
                        .distinct()
                        .collect(Collectors.toList());
```

forEach(): Performs an action for each element.

```
myList.stream().forEach(System.out::println);
```

collect(): Accumulates elements into a collection.

```
List<String> collected = myList.stream()
                         .collect(Collectors.toList());
```

count(): Returns the number of elements.

```
long count = myList.stream().count();
```

flatMap() : Transforms each element into a stream of elements and then flattens these streams into a single stream.

```
List<List<Integer>> nestedList = Arrays.asList(
   Arrays.asList(1, 2),
```

```
    Arrays.asList(3, 4),
    Arrays.asList(5, 6)
);
```

```
List<Integer> flattenedList = nestedList.stream()
                                .flatMap(List::stream)
                                .collect(Collectors.toList());
```

Collectors.groupingBy(): Groups elements based on a classifier function.

Collectors.partitioningBy(): Partitions elements into two groups based on a predicate.

```
Map<Integer, List<String>> groupedByLength = myList.stream()
                                        .collect(Collectors.groupingBy(String::length));
```

```
Map<Boolean, List<String>> partitioned = myList.stream()
                                    .collect(Collectors.partitioningBy(s -> s.length() > 4));
```

Stream.concat(): Concatenates two streams.

```
Stream<String> stream1 = Stream.of("apple", "banana");
Stream<String> stream2 = Stream.of("orange", "kiwi");

Stream<String> combinedStream = Stream.concat(stream1, stream2);
```

Java 8 introduced specialized streams for primitive types to avoid boxing/unboxing overhead.

```
IntStream intStream = IntStream.range(1, 5);
DoubleStream doubleStream = DoubleStream.of(1.0, 2.0, 3.0);
```

Built-In Java Functional Interfaces:

- Runnable: This interface only contains the run() method.
- Comparable: This interface only contains the compareTo() method.
- ActionListener: This interface only contains the actionPerformed() method.

- Callable: This interface only contains the call() method.

Types of Functional Interfaces in Java:

1. Consumer
2. Predicate
3. Function
4. Supplier

1. Consumer

1. The consumer interface of the functional interface is the one that accepts only one argument.
2. The consumer interface has no return value.
3. It returns nothing.
4. Consumer<Integer> consumer = (value) -> System.out.println(value);

2. Predicate

1. The Predicate interface represents a boolean-valued function of one argument.
2. It is commonly used for filtering operations in streams.
3. Predicate predicate = (value) -> value != null;

3. Function

1. A function is a type of functional interface in Java that receives only a single argument and returns a value after the required processing.
2. Many different versions of the function interfaces are instrumental and are commonly used in primitive types like double, int, long.
3. Function<Integer, Integer> function = (value) -> value * value;
4. The Bi-Function is substantially related to a Function. Besides, it takes two arguments, whereas Function accepts one argument.

4. Supplier:

1. The Supplier functional interface is also a type of functional interface that does not take any input or argument and yet returns a single output.
2. Supplier<String> supplier = () -> “Hello, World!”;

Spring Framework

Spring:

1. Spring framework is employed in developing Java applications and provides a set of modular, reusable components specifically designed for enterprise-level software development.
2. Dependency Injection:

- It allows the developers to implement Inversion of Control (IoC).
- It allows loose coupling, and flexibility, and facilitates the easier testing of objects.

3. Declarative Configuration

- It allows configuration through XML or annotations which provides a declarative approach for setting up application components
- It also helps in reducing boilerplate code.

4. Integrated Security

- It offers a security framework, authentication, authorization, and other security features.
- It gives a very simple way to build Java applications.

5. Aspect-Oriented Programming (AOP)

- It separates the coding aspects concerns from the core business logic.
- It helps in modularizing concerns like logging, transactions, and security.

Spring Boot:

1. It is a framework that is designed to simplify the process of building and deploying production-ready applications.
2. It also provides conventions and defaults for application setup and reduces the need for manual configuration.

3. It is Java-based web, microservices, and enterprise applications that promote efficiency and allow developers to focus on business logic rather than infrastructure concerns which makes loosely coupled software applications so that modules are independent and further modifications become easier.
4. Embedded Servers: It supports embedded servers like Tomcat, Jetty, or Undertow. It allows the applications to run as standalone JARs and simplifies the deployment.
5. Spring Boot Starters: They are ready-made templates that include dependencies, configuration, and accelerating the setup of common application features like web, data, and security**.** It automatically ensures that added dependencies are compatible and up-to-date.
6. Auto-Configuration: Spring boot automatically configures the spring beans based on the dependencies in the individual classes. It reduces the need for explicit configuration of the application and boilerplate code.

Spring MVC:

1. It helps in the development of web applications by considering the Model-View-Controller (MVC) architectural pattern.
2. It offers a structured approach to the code design, also promoting the creation of robust and scalable web applications.
3. The core of the Spring MVC Framework is the Dispatcher Servlet which helps in the functioning as the Front Controller by managing the incoming Http Requests by directing them to the relevant controller.

1. MVC Architecture

- It follows the Model-View-Controller (MVC) pattern which promotes a modular and organized structure for building web applications.
- It provides a lot of in-built annotations which reduces the code length and helps to build scalable software applications.

2. Annotation-Based Configuration

- It supports the use of annotations for configuration which reduces the XML-based configuration.
- It makes the development more accurate.

3. Powerful Data Binding

- It has robust data binding capabilities which simplifies the handling of such formed data.
- It makes more easier to bind the incoming requests in a single unit within the Java objects.

4. Flexible Handler Mapping

- It maps incoming requests to the appropriate controller methods.
- It maps based on various criteria such as URL patterns, HTTP methods, headers, and parameters which provides flexibility in request handling.

Feature	Spring Framework	Spring MVC	Spring Boot
Type	Core application framework	Web framework module of Spring	Rapid development framework built on Spring
Purpose	Provides infrastructure for Java apps	Used to build web (MVC/REST) applications	Simplifies Spring development with auto-config
Web Capabilities	Not web-specific	Yes – for building web apps	Yes – embedded server support, REST APIs, etc.
Configuration	Manual (XML or Java Config)	Manual – DispatcherServlet, ViewResolver setup	Auto-configured via annotations and properties
Embedded Server	❌ No	❌ No	✅ Yes (Tomcat, Jetty, etc.)
Starter Dependencies	❌ No	❌ No	✅ Yes (spring-boot-starter-web, etc.)
Main Class Support	❌ No	❌ No	✅ Yes – @SpringBootApplication

Feature	Spring Framework	Spring MVC	Spring Boot
Deployment	WAR files, external server	WAR files, external server	Runnable JAR with embedded server

Exception Handling in Spring Boot:

Use Specific Exceptions Over Generic Ones:

1. A clean coder avoids using generic exceptions
   like Exception or RuntimeException because they provide no meaningful information about the error.
2. Always use more specific exceptions.
3. In Spring Boot, you can create custom exceptions to represent specific error conditions.
4. Handle Exceptions Using @ControllerAdvice
5. Spring Boot offers @ControllerAdvice as a centralized exception handler, making your code more organized and maintainable.
6. It allows you to catch exceptions globally and return custom error responses without cluttering business logic with error handling.
7. The @ExceptionHandler annotation tells Spring Boot that the method should handle specific exceptions.

Service Discovery:

1. Service discovery is the process by which components in a distributed system locate and communicate with each other.
2. In the context of microservices, where numerous small, independent services collaborate to deliver an application's functionality, service discovery becomes paramount.

3. Microservices are loosely coupled, meaning they can be deployed independently and communicate over a network.
4. These services need to discover and communicate with each other dynamically, as their locations (IP addresses and ports) may change due to scaling, deployment updates, or failures.
5. Service discovery enables this dynamic interaction by providing a centralized mechanism for service registration and lookup.
6. Registration:
    a. When a microservice starts up, it registers itself with the service discovery mechanism, typically by providing it, a network location (IP address and port) and other metadata.
    b. This registration process allows the service discovery system to maintain an up-to-date catalogue of available services and their locations.
7. Lookup**:**
    a. When a microservice needs to communicate with another service, it queries the service discovery system, providing the name or identifier of the target service.
    b. The service discovery system then resolves this query by returning the network location of the requested service.
    c. With this information, the calling service can establish a connection and communicate with the target service.

API Gateway:

1. The API Gateway Pattern in some cases stands for "Backend for frontend".
2. It is basically the entry gate for taking entry into any application by an external source.
3. API gateway simplifies the communication between a client and a service, whether that be between a user's web browser and a server, or

between a frontend application and the backend application that it relies on.

4. Spring Cloud Gateway provides a library for making API gateways on top of Spring and Java.
5. It provides a flexible way of routing requests based on a number of criteria, as well as focuses on cross-cutting problems like security, resiliency, and monitoring.
6. Some of the important features of Spring Cloud Gateway are:
    a. It is Built on Spring Framework 5, Project Reactor, and Spring Boot 2.0
    b. You can integrate Circuit Breaker with Spring Cloud Gateway.
    c. You can integrate Spring Cloud DiscoveryClient.
    d. Predicates and filters are specific to routes.
    e. Path Rewriting

Resilience 4j:

Main Patterns in Resilience4j

- Circuit Breaker
- Retry
- Rate Limiter
- Retry
- Bulkhead
- Time Limiter

Retry:

1. In Spring Boot, a retry mechanism is used to automatically re-execute a failed operation (such as calling an external API or database) when certain exceptions occur.
2. This improves fault tolerance and resilience in your application.

3. You use retry when: A temporary failure may succeed if retried (e.g., network issues, rate-limiting, timeouts). You want to avoid manual retry logic (using loops or try-catch).
4. **maxAttempts:** the maximum number of attempts, including the initial call, default is 3.
5. **waitDuration:** the fixed time between attempts, default is 500 milliseconds.

Rate Limiter:

1. A Rate Limiter is a resilience pattern used to control the rate of requests sent to a service or resource.
2. It helps prevent overload by limiting the number of requests allowed within a specific time period.
3. The main goal of a Rate Limiter is to ensure that the system can handle incoming requests without being overwhelmed, thus improving its stability and performance.
4. limit-for-period: Max allowed calls per window
5. limit-refresh-period: Window time
6. timeout-duration: Wait time if limit exceeded

Time Limiter:

1. Time Limiter is a Resilience4j module used to enforce a maximum time limit for executing a method, especially useful for remote calls like APIs, database queries, or service-to-service calls.
2. If the operation takes longer than the allowed time, it's automatically interrupted and a fallback can be triggered.
3. Prevent long-running or stuck operations from consuming system resources.
4. Fail fast and return fallback responses quickly.
5. Improve system responsiveness and resilience.

Bulkhead:

1. In resilience engineering, a Bulkhead pattern is used to isolate different parts of the system so that a failure or overload in one component doesn't bring down the entire system.
2. In Spring Boot, we implement this pattern using Resilience4j's Bulkhead module, which allows us to limit the number of concurrent calls to a particular method or service.
3. To prevent resource exhaustion (e.g., threads, memory).
4. To isolate system failures.
5. To ensure critical operations remain available even if others are overloaded.

Circuit Breaker:

1. A Circuit Breaker is a fault-tolerance pattern that prevents a system from repeatedly trying to execute an operation that is likely to fail — typically due to downstream service issues (e.g., timeout, connection errors, etc.).
2. It helps to:

- Avoid system overload
- Fail fast and return fallback responses
- Improve resilience and stability

How Circuit Breaker Works?

State	Description
Closed	All requests are allowed. Failures are counted.
Open	No requests are allowed. Immediate fallback.
Half-Open	Some trial requests are allowed. If they succeed, the circuit closes. Otherwise, it opens again.

If your microservice calls an external REST API and that service goes down, using a Circuit Breaker:

- Prevents your app from making repeated calls to the failing API.

- Reduces load on the external service.
- Quickly returns a fallback response to the client.